CLASE 7 · PUENTE AL APRENDIZAJE PROFUNDO

Redes convolucionales

Imágenes, kernels, detección de bordes y cómo se aprende un filtro

github.com/datahuba/curso-redes-neuronales · Basado en el curso del Dr. J. Irving Vasquez (jivg.org)

Agenda de hoy

Tiempo	Tema
0:00	Repaso
0:10	Imágenes y por qué convolución
0:25	Correlación cruzada
0:50	Detector de bordes y ReLU
1:00	Práctica: 3_1
1:30	Descanso
1:40	Aprender el kernel
2:05	Reto: 3_2 y practica_07
2:25	Quiz y cierre

Al terminar la clase podrás…

1

Explicar

por qué se usan convoluciones en imágenes.

2

Calcular

una correlación cruzada a mano.

3

Programar

la operación de una capa convolucional.

4

Entrenar

los números de un kernel con descenso.

Repaso

¿Una capa?

$H = X \cdot W + B$ y una activación.

¿Cómo se entrena?

Descenso por gradiente.

¿Momentum?

Suaviza rebotes y permite η grande.

Hoy: lo mismo, pero sobre imágenes.

Una imagen es una matriz de números

30	60	120	200	240	250
25	55	110	190	235	245
20	50	100	180	230	240
20	45	95	170	225	235
15	40	90	160	220	230

Escala de grises: 0 = negro, 255 = blanco.

A color: 3 matrices (rojo, verde, azul).

La foto de los notebooks: 256 × 256 = 65 536 números por canal.

¿Por qué no una capa densa?

Capa densa

65 536 píxeles × 100 neuronas

= 6.5 millones de pesos

Ignora que los píxeles vecinos se relacionan.

Convolución

Un filtro de 3 × 3

= 9 pesos

Los mismos pesos recorren toda la imagen: buscan el mismo patrón en todas partes.

Un gato en la esquina o en el centro: el mismo filtro lo detecta.

El kernel se desliza sobre la imagen



1. Coloca el kernel.
2. Multiplica cada número por el píxel de abajo.
3. Suma todo: un píxel de salida.
4. Desliza y repite.

Ejemplo en el pizarrón

Cada fila de la imagen: `0 0 9 9 9` · cada fila del kernel: `−1 0 1`

Ventana (columnas)	Valores	Cálculo por fila	× 3 filas
0 – 2	0 0 9	−0 + 0 + 9 = 9	27
1 – 3	0 9 9	−0 + 0 + 9 = 9	27
2 – 4	9 9 9	−9 + 0 + 9 = 0	0 (zona plana)

Salida: `[[27, 27, 0], [27, 27, 0], [27, 27, 0]]`

El kernel se enciende donde hay un borde vertical y vale 0 donde la imagen es plana.

Tamaño de la salida

$$\text{sin padding:}\quad \text{salida} = n - 2k$$

Imagen	Kernel	k	Salida
5 × 5	3 × 3	1	3 × 3
256 × 256	3 × 3	1	254 × 254
256 × 256	5 × 5	2	252 × 252

Con padding (rellenar la orilla con ceros) la salida puede medir lo mismo que la entrada.

El kernel del notebook 3_1

−1/3	0	1/3
−1/3	0	1/3
−1/3	0	1/3

Positivo donde la imagen pasa de oscuro a claro (→).

Negativo donde pasa de claro a oscuro.

≈ **0** en zonas planas.

¿Por qué solo se ven algunos bordes?

La ReLU borra negativos

Los bordes de claro a oscuro dan valores negativos: la ReLU los convierte en 0.

Solo compara izquierda y derecha

Los bordes horizontales no cambian de izquierda a derecha: este kernel no los ve.

Es la respuesta a la pregunta del notebook 3_1.

NOTEBOOKS · 30 MIN · 4 PUNTOS

Práctica · 3_1_capa_convolucional

TODO 1 pt correlacionPixel: dos for de −k a k

TODO 1 pt G de ceros del tamaño de I

TODO 1 pt recortar las orillas: G[k:m-k, k:n-k]

1 pt pregunta de la ReLU

Tras la ReLU se ven solo algunos bordes verticales, como en files/ccnout.png.

```
# comprobaciones
imagen: (256, 256) uint8
salida: (254, 254)

# resultado debe ser de NumPy:
resultado = np.array(sumatoria)
```

Descanso · 10 min

Después: aprender el kernel

Los números del kernel se aprenden

w_1	w_2	w_3
w_4	w_5	w_6
w_7	w_8	w_9

En una red convolucional nadie elige los números del kernel.

Son los pesos de la capa: se aprenden con descenso por gradiente.

Igual que en la clase 4, pero ahora la “predicción” es una imagen entera.

El gradiente de un peso del kernel

$$\partial ECM/\partial K[u,v] = (2/N) \cdot \Sigma\ (P - T) \cdot [G > 0] \cdot I[i+u,\ j+v]$$

Pieza	Significado
P − T	error en cada píxel (predicción − objetivo)
[G > 0]	1 si la ReLU dejó pasar, 0 si no (derivada de la ReLU)
I[i+u, j+v]	el píxel que se multiplicó por ese peso

Luego: $K \leftarrow K - \eta \cdot \text{gradiente}$

Trampas y trucos

Trampas

- **ReLU muerta:** todo G negativo → sin gradiente.
- **Escala:** píxeles 0–255 → gradientes enormes.
- **Lentitud:** píxel por píxel tarda minutos.

Trucos

- **Normalizar:** dividir entre 255.
- **Centrar:** restar el promedio.
- **Vectorizar** y usar **momentum**.

Vectorizar: versiones desplazadas

```
# una versión desplazada por cada peso (u, v)
d = {(u, v): I[k+u:m-k+u, k+v:n-k+v]
     for u in range(-k, k+1) for v in range(-k, k+1)}

G = sum(K[u+k, v+k] * d[(u, v)] for (u, v) in d)
```

La correlación es la suma de cada versión por su peso.

El gradiente de cada peso es un promedio sobre una matriz completa.

Milisegundos por época.

Resultado: el descenso descubre el filtro de Sobel

Sobel (objetivo)

-1	-2	-1
0	0	0
1	2	1

Aprendido (reto 3_2)

−0.86	−1.77	−0.89
−0.03	−0.01	0.04
0.93	1.79	0.84

practica_07: recupera Sobel exacto (ECM 0.97 → 0).

Reto 3_2: ECM 31 600 → 29; la imagen predicha es casi idéntica.

Una CNN profunda hace esto con muchos kernels y muchas capas: bordes → formas → objetos.

NOTEBOOKS · 20 MIN · RETO (PUNTOS EXTRA)

Práctica · 3_2 · reto + practica_07

1. Normaliza y centra la entrada
2. Vectoriza con versiones desplazadas
3. Descenso con momentum β 0.9, η 2, 2000 épocas

Guía con la estrategia completa:
docs/guias_notebooks/NB_3_2.

```
# referencia del reto
ECM inicial ≈ 31600
ECM final   ≈ 29
kernel ≈ Sobel en el centro

# demo:
# python docs/practica_07.py --graficar
```

Quiz · 5 minutos

1. ¿Por qué no usar capas densas para imágenes?
2. ¿Cuánto mide la salida de una imagen 7×7 con un kernel 3×3?
3. ¿Qué detecta el kernel con filas $-1\ 0\ 1$?
4. ¿Por qué se ven solo algunos bordes tras la ReLU?
5. ¿Qué son los “pesos” de una capa convolucional?

Para llevar

1. Convolución: un filtro pequeño recorre la imagen con los mismos pesos.
2. Los kernels detectan patrones y se aprenden con descenso por gradiente.
3. Las redes profundas apilan muchas capas convolucionales.

Fin del curso introductorio. Siguiente: aprendizaje profundo.

Tarea: notebook 3_1 (4 pts) + reto 3_2 (extra)

Leer: Goodfellow et al., Deep Learning, cap. 9 (redes convolucionales)